X — Backend Implementation

X.1 Overview

X.1.1 Objectives

The Chairpal backend serves as the centralized system orchestrating communication between the Flutter mobile application, IoT wheelchair hardware, and the MySQL database. Its primary objectives are:

- **API Provisioning:** A comprehensive suite of RESTful APIs for secure, seamless data flow to the Flutter client.
- **Hierarchical Spatial Management:** Managing the indoor navigation hierarchy: Organization → Building → Floor → Place.
- **Indoor Maps & Semantic Data:** Storing spatial (X, Y) coordinates and accessibility metadata for indoor points of interest.
- **IoT Telemetry Integration:** Serving as the cloud endpoint for continuous wheelchair sensor streaming (movement, obstacles, health).
- **Real-Time Event Processing:** Broadcasting critical events (live location, fall detection SOS) to connected clients via WebSockets.

X.1.2 Responsibilities

- **Business Logic:** Enforcing trip lifecycles, connection limits between patients and doctors, and sensor event deduplication.
- **Data Management:** Maintaining referential integrity via Eloquent ORM within a normalized MySQL schema.
- **Authentication & Authorization:** Dual-layered security separating human user authentication from IoT machine-to-machine authentication.
- **Real-Time Data Handling:** Managing concurrent, high-frequency updates from multiple wheelchairs simultaneously.

X.2 System Architecture

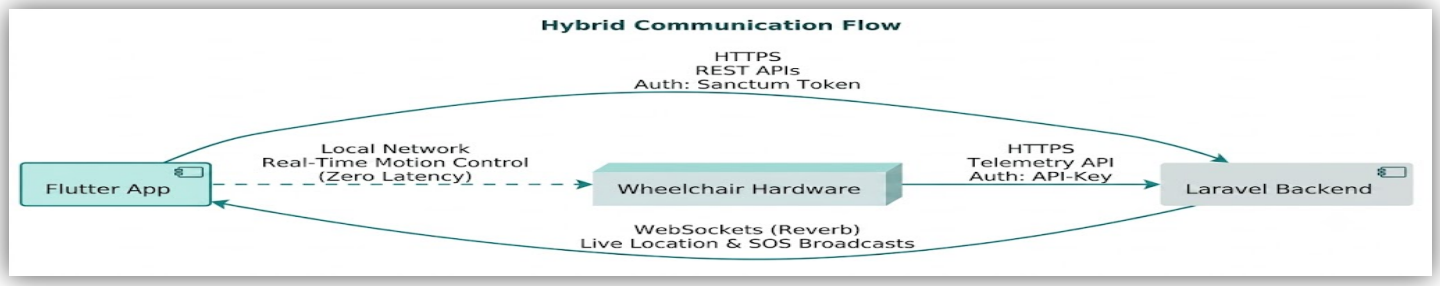
X.2.1 Components

1. **Flutter Mobile Application** — Client interface for Patients, Companions, Doctors, and Organization Admins.
2. **Laravel Backend API** — Core cloud infrastructure hosting business logic, spatial mapping, and user data.
3. **MySQL Database** — Relational database managing persistent state.
4. **Wheelchair Hardware (IoT)** — Physical wheelchair running ROS/Python, streaming sensor data to the backend.

X.2.2 Communication Flow (Hybrid Approach)

The system uses a hybrid approach to guarantee safety and minimize latency:

- **User ↔ Backend:** Standard HTTPS REST for dashboards, profiles, community features, and chatbots.
- **Wheelchair ↔ Backend:** Dedicated HTTPS telemetry channel authenticated via API Key, pushing continuous status updates.
- **Flutter ↔ Wheelchair (LAN):** Motion control commands (e.g., joystick) are sent directly over the local network to eliminate cloud latency. The backend acts strictly as a telemetry observer, not a control relay.



X.2.3 Architectural Style

The backend follows **Service-Oriented Architecture (SOA)** with decoupled, modular services. It adheres to **RESTful API** design principles (standard HTTP verbs and status codes) and incorporates **Event-Driven Architecture (EDA)** for real-time functionality — critical events such as a fall trigger asynchronous listeners.

X.3 Technology Stack

Component	Technology	Rational
Backend Framework	Laravel	Expressive syntax, robust security, built-in Eloquent ORM, WebSocket support
Language	PHP 8+	Strong typing, enums, match expressions
Database	MySQL	ACID compliance for complex spatial and user relationships
Real-Time	Laravel Reverb (WebSockets)	Server-push for live GPS/indoor coordinates and SOS alerts
ORM	Eloquent	Abstracts complex SQL; manages relationships efficiently
File Storage	Laravel localized storage	Manages avatars, place images, and floor map JSON/images

X.4 Authentication & Authorization

X.4.1 Dual Authentication Model

1. **Human Users (Flutter):** Laravel Sanctum. Users authenticate via email or username; the backend issues a Bearer Token for all subsequent requests.
2. **Wheelchair (IoT):** API Key Authentication. Upon initial connection via Flutter, the backend generates a secure 32-character `api_key` bound to the wheelchair's serial number. Flutter passes this key locally to the wheelchair, which attaches it to a custom `api-key` header on all telemetry requests.

X.4.2 Custom Middleware: `VerifyWheelchairApiKey`

IoT endpoints are secured via dedicated middleware that validates the API key and attaches the resolved wheelchair object to the request, eliminating redundant database queries in controllers.

```php
public function handle(Request $request, Closure $next): Response
{
    $apiKey = $request->header('api-key');
    if (!$apiKey) {
        return response()->json(['message' => 'Missing api-key header.'], 401);
    }
    $wheelchair = Wheelchair::where('api_key', $apiKey)->first();
    if (!$wheelchair) {
        return response()->json(['message' => 'Invalid api-key.'], 401);
    }
    // Attach wheelchair to request so we can access it in controller without re-querying
    $request->merge(['authenticated_wheelchair' => $wheelchair]);
    return $next($request);
}
```

X.4.3 Role-Based Access Control (RBAC) & Community Rules

The system manages four roles: `user` (Patient), `companion`, `doctor`, and **`org_admin`**. Connections are formed via `POST /api/community/friends/send` and remain `pending` until accepted. Role constraints are strictly enforced:

- **Companion:** Can follow exactly **1 Patient**; a Patient may have multiple Companions.
- **Doctor:** A Patient may have only **1 active Doctor**; a Doctor may manage multiple patients.
- **Access Separation:** Only accepted Doctor/Companion relationships unlock sensitive health dashboards and live location streams.

X.4.4 Role-Based Dashboards

The `DashboardController` dynamically aggregates data based on the authenticated user's role:

1. **Patient (user):** Displays current vitals (heart rate, temperature, tilt angle), graphical trends, recent AI alerts, and last visited places.

Sample response:

```json
{
  "message": "User dashboard data retrieved.",
  "data": {
    "current_vitals": {
      "heart": { "value": 85, "status": "normal" },
      "obstacle": { "movement": "movement", "mpu_angle": 1.5, "status": "normal" }
    },
    "ai_recommendation": {
      "risk_level": "medium",
```

```
    "recommendation": "Maintain steady pace. Slight incline detected."
  },
  "last_visited_places": [
   { "id": 4, "name": "Radiology Lab" }
  ]
 }
}
```


2. **Doctor:** Statistical overview of all supervised patients, categorized by risk level (Normal / Medium / Critical), with access to individual patient health details.
3. **Companion:** Returns the assigned patient's data and activates Live Tracking via Reverb WebSockets (`private-wheelchair.{id}.location`), showing the patient moving on the indoor map in real time.
4. **Organization Admin (`org_admin`):** Manages the spatial hierarchy — adding Buildings, Floors, uploading Map JSON, and assigning coordinates to Places (e.g., x:10, y:25 → "Cafeteria").

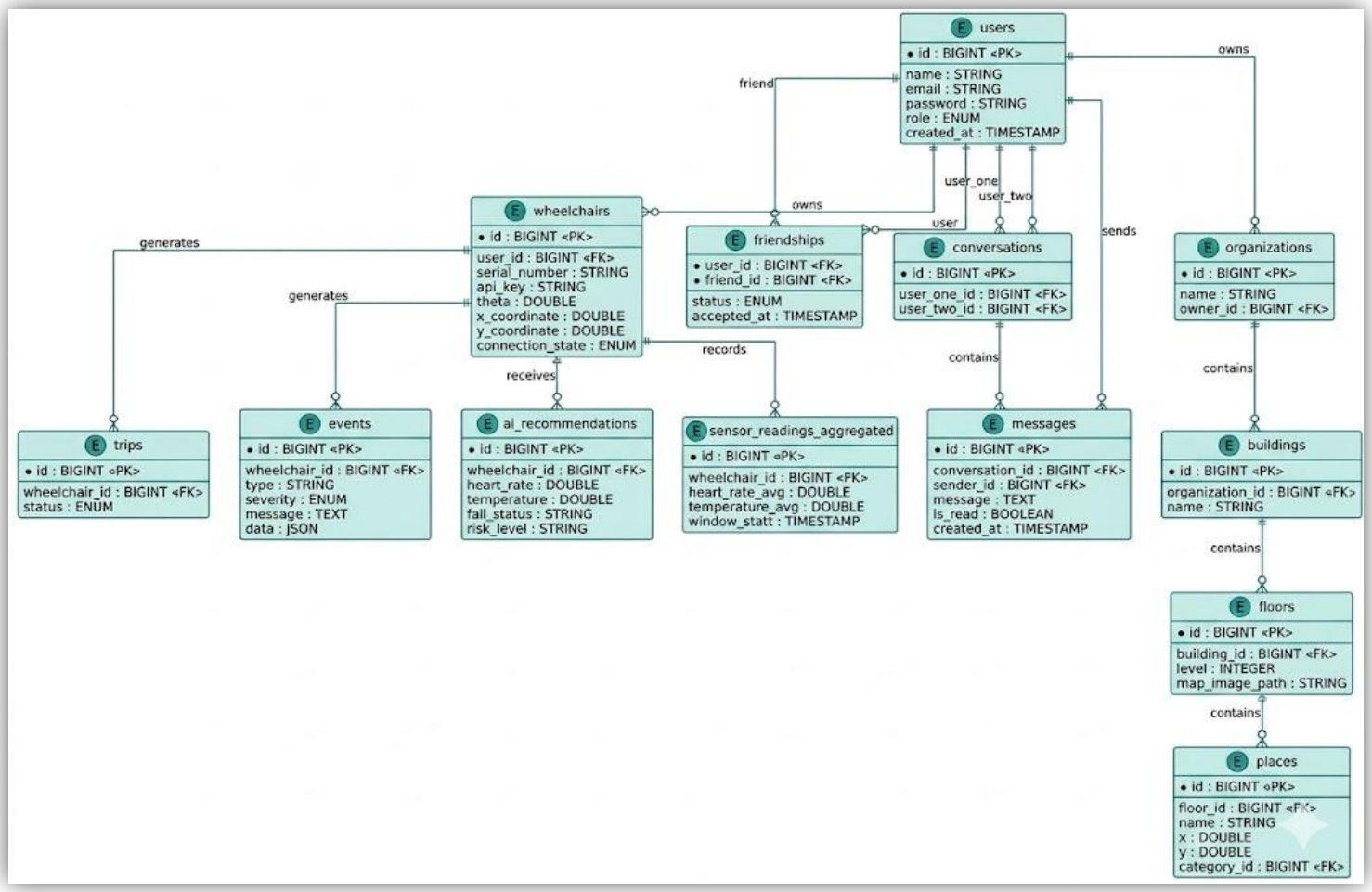
X.4.5 Authorization Policies

Laravel Policies enforce granular resource security. For example, `PlacePolicy` ensures only the Organization Admin who owns a building can edit or delete places within it, preventing cross-tenant data access.

X.5 Database Design

The database targets **3rd Normal Form (3NF)** and is optimized for high-frequency IoT data, spatial hierarchies, and strict medical relationships.

X.5.1 ERD — Main Entities



X.5.2 Key Relationships

Patient ↔ Companion Connection

- Companion sends a request; a Companion can follow exactly 1 Patient.

- On acceptance: Companion is assigned; a notification and email are dispatched; a Chat is automatically created; Companion gains Community access and inherits shared access to all spatial records (Organizations, Buildings, Floors, Maps, Places) the Patient has access to, including add/edit/delete rights.

Patient ↔ Doctor Connection

- Patient sends a medical request; a Patient can have only 1 Doctor.
- On acceptance: Doctor becomes medical supervisor; a notification and email are dispatched to the Patient; a Chat is created; Doctor gains access to the Doctor Dashboard (risk-level statistics and per-patient vitals/AI recommendations).

X.5.3 Data Integrity & Sensor Pruning

- **Normalization:** Adheres to 1NF (atomic values), 2NF (no partial composite-key dependencies), 3NF (no transitive dependencies). Spatial hierarchy enforces this: Place → Floor → Building.
- **Sensor Pruning:** A scheduled Laravel Cron Job deletes `sensor_readings_aggregated` records older than 30 days nightly at 02:00 to prevent database bloat.

```php
// In app/Console/Kernel.php
protected function schedule(Schedule $schedule)
{
  // Delete sensor reading data older than 30 days every night to maintain Data Integrity
  $schedule->call(function () {
     \DB::table('sensor_readings_aggregated')
        ->where('window_start', '<', now()->subDays(30))
        ->delete();
  })->dailyAt('02:00');
}
```

X.6 Hierarchical Location Architecture

The backend enforces a strict **Organization → Building → Floor → Place** hierarchy, ensuring all spatial data is consistently organized for seamless wheelchair navigation across indoor environments.
Each Place (e.g., "Cardiology Room") stores precise floating-point (x, y) coordinates relative to its parent Floor's image map, bridging human-readable locations with robotic coordinate systems.

X.7 Indoor Navigation Support

- **Map Delivery:** Secure endpoints serve high-resolution floor map images used by both Flutter and the wheelchair hardware as navigational grids.
- **Semantic Spatial Queries:** The API supports querying places by category or proximity, generating dynamic destination lists.
- **Location Context:** The backend aligns the user's current Floor with the wheelchair's relative (X, Y) coordinates to calculate distances to nearby Places.

X.8 Wheelchair IoT Integration

X.8.1 Device Provisioning

1. Flutter submits the hardware's `serial_number`.
2. Backend generates a cryptographic 32-character `api_key` and registers the device.
3. Flutter receives the key and passes it locally to the wheelchair.

X.8.2 Payload Optimization

The `wheelchair_id` is omitted entirely from request bodies. The `VerifyWheelchairApiKey` middleware resolves the device from the `api-key` header alone, reducing bandwidth and processing overhead on constrained IoT hardware.

X.9 Real-Time Monitoring & Emergency Protocols

X.9.1 Continuous Telemetry

Telemetry arrives via `POST /api/trip/movement/update`. The backend updates the wheelchair's absolute (x, y, theta) coordinates and broadcasts them over WebSockets (Reverb) to Flutter for live tracking.

X.9.2 Event Deduplication

To prevent database flooding from sensors stuck in error states (e.g., the same obstacle detected 50 times per second), `Event::findDuplicate()` updates the timestamp of an existing event rather than inserting duplicate rows.

X.9.3 Emergency SOS Protocol

When the AI detects a critical anomaly (e.g., a sudden `mpu_angle` drop indicating a fall):

1. A high-priority database notification and WebSocket alert are dispatched to the assigned Companion.
2. An emergency message is automatically injected into the active Companion–Patient Chat in the Community module.
3. The message includes a payload with exact coordinates and a live-location trigger.

Sample SOS chat injection:

```
{
 "sender": "System_Emergency",
 "message": " 🚨 AUTOMATIC SOS: Patient has experienced a critical fall!",
 "payload": {
  "latitude": 30.0444,
  "longitude": 31.2357,
  "status": "CRITICAL",
  "live_location_active": true
 },
 "timestamp": "2026-06-11T13:40:00Z"
}
```

X.10 API Design

X.10.1 RESTful Principles & Standardized Responses

A custom `ApiController` enforces a uniform JSON contract across all endpoints, guaranteeing consistent `success`, `message`, and `data/errors` fields regardless of outcome.

```php
// Inside ApiController.php
protected function successResponse($message, $code = 200, $parameters = [])
{
  return response()->json(array_merge([
     'success' => true,
     'message' => $message,
  ], $parameters), $code);
}
protected function errorResponse($message, $code = 400)
{
  return response()->json([
     'success' => false,
     'message' => $message,
  ], $code);
}
```

X.10.2 API Resources (Data Transformation)

Raw database output passes through a transformation layer before reaching the client, stripping sensitive fields (e.g., passwords), formatting dates via Carbon, and computing dynamic attributes such as `average_rating`.

X.10.3 Dynamic Includes

To minimize client-side round trips, computed attributes (e.g., `visitors_count`, `rating_distribution`) are appended directly to resource responses.

Sample transformed Place payload:

```
{
 "success": true,
 "message": "Place retrieved successfully!",
 "data": {
  "id": 1,
  "name": "Cardiology Department",
  "floor_id": 2,
  "average_rating": 4.5,
  "visitors_count": 120,
  "rating_distribution": {
   "1": "0%", "2": "0%", "3": "10%", "4": "30%", "5": "60%"
```

```
  }
 }
}
```

X.11 File Management

Floor maps (PNG/JPG), user avatars, and place images are stored on Laravel's public storage disk (`storage/app/public/maps`). Relative paths are saved in the database and resolved to full URLs at query time. All upload endpoints enforce MIME type validation and a 2 MB size limit (`image|mimes:jpeg,png,jpg,gif,svg|max:2048`) to prevent storage exhaustion.

X.12 Performance Optimization

X.12.1 Eager Loading (N+1 Prevention)

Eloquent's `with()` resolves related models in a single optimized SQL query, avoiding the N+1 problem on data-heavy views such as the Doctor dashboard.

```
// In DashboardController.php
$activeTrip = Trip::with(['place', 'wheelchair'])
  ->whereHas('wheelchair', function ($q) use ($targetUser) {
    $q->where('user_id', $targetUser->id);
  })->whereIn('status', ['started', 'in_progress'])->latest()->first();
```

X.12.2 Pagination

Large datasets (Community Posts, Friends Lists, Place Reviews) are paginated at 15 records per request (**`->paginate(15)`**), preventing memory overflow on both server and client.

X.13 Error Handling & Logging

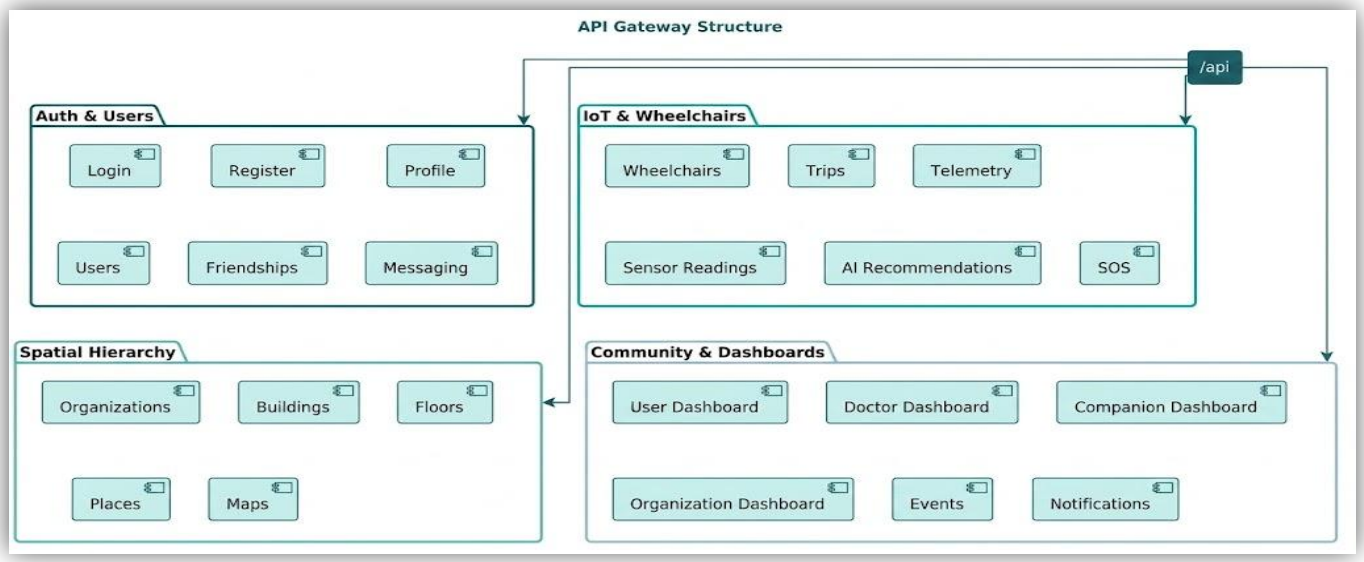
X.13.1 Form Request Validation

All incoming requests are validated by dedicated `FormRequest` classes. Malformed payloads are rejected immediately with a `422 Unprocessable Entity` response detailing the failing fields, keeping controllers clean.

X.13.2 Global Exception Handling & Audit Logs

- **Exception Handling:** Database failures and unauthorized access are caught in `try-catch` blocks. Transactions are rolled back (`DB::rollBack()`) to prevent partial data corruption; a clean `500 Internal Server Error` is returned.
- **Audit Logging:** Critical actions (e.g., a Patient removing a Companion or Doctor) are permanently logged to an `AuditLog` table, capturing the action, targeted user, IP address, and timestamp.

X.14 REST API Endpoint Reference



X.14.1 Authentication & Profile

Method	Endpoint	Auth	Description
POST	/api/signup	None	Registers a new user (Patient, Doctor, Companion) and creates medical profile.
POST	/api/login	None	Authenticates user and returns Sanctum Bearer Token.
POST	/api/logout	Sanctum	Revokes the current access token.
GET	/api/user/profile	Sanctum	Retrieves the authenticated user's profile and medical conditions.
PUT	/api/user/profile	Sanctum	Updates user profile data (height, weight, etc.).

X.14.2 Hierarchical Spatial Navigation

Method	Endpoint	Auth	Description
GET	/api/organizations	Sanctum	Lists available hospital/organization campuses.
POST	/api/buildings	Sanctum (Admin)	Creates a new building within an organization.
POST	/api/floors	Sanctum (Admin)	Creates a floor and uploads its map image.
GET	/api/floors/{id}/places	Sanctum	Fetches all navigable places (rooms, labs) on a specific floor.
POST	/api/places	Sanctum (Admin/Comp)	Adds a new semantic place with (x, y) coordinates to a floor.

X.14.3 IoT & Wheelchair (API-Key Protected)

Method	Endpoint	Auth	Description
POST	/api/wheelchairs/connect	Sanctum	Flutter requests connection. Backend returns a secure api_key.
POST	/api/trip/movement/update	api-key	Hardware streams absolute (x, y, theta) coordinates. Broadcasts to UI.
POST	/api/wheelchair/health	api-key	Hardware streams Vitals & Fall Status. Triggers automatic SOS if critical.
POST	/api/trip/events	api-key	Hardware logs obstacles or warnings. Analyzed and deduplicated.

X.14.4 Trip Management

Method	Endpoint	Auth	Description
POST	/api/trips/start	Sanctum	Initiates a navigation trip. Links wheelchair to destination place.
POST	/api/trips/{tripId}/end	Sanctum	Closes an active trip and calculates trip duration/statistics.
GET	/api/trips	Sanctum	Retrieves historical trip records for the user.

X.14.5 Community & Dashboards

Method	Endpoint	Auth	Description
POST	/api/community/friends/send	Sanctum	Sends a Companion/Doctor request with strict role validation.
POST	/api/community/friends/requests/{id}	Sanctum	Accepts/Rejects connection. Auto-generates Chat & Notifications upon acceptance.
GET	/api/dashboard/user	Sanctum	Fetches real-time Patient vitals, active trips, and recent alerts.
GET	/api/dashboard/doctor	Sanctum	Aggregates health statistics and risk levels for all supervised patients.
GET	/api/dashboard/companion	Sanctum	Fetches target patient data and enables Live WebSockets tracking.